Expert Systems With Applications 216 (2023) 119437

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

Data complexity-based dynamic ensembling of SVMs in classification

Sowkarthika B. *, Manasi Gyanchandani, Rajesh Wadhvani, Sanyam Shukla

MANIT, Bhopal, Madhya Pradesh 462007, India

ARTICLE INFO

Keywords:
Data complexity
Classification model
Minimum spanning tree
Ensemble

ABSTRACT

Ensemble-based techniques are deployed to yield better performance than individual classifiers. Existing ensembling approaches fail to consider data complexity during their design. This work presents an ensemble-based approach for resolving complex patterns in real-world classification problems. A novel Minimum Spanning Tree (MST)-based approach is employed for decomposing the original problem into subproblems with reduced data complexity, and SVM is utilized for the development of component classifiers for these subproblems. A novel dynamic ensemble-based technique is utilized to generate the outcome. This work is evaluated by using 28 datasets retrieved from the KEEL dataset repository. Additionally, statistical tests are performed to illustrate a significant difference in the performance of the proposed model compared to the state-of-art classification models and two recently proposed dynamic ensembling approaches.

1. Introduction

A classification algorithm recognizes and systematically arranges the training data into preset categories for a given dataset. Classification finds its application in numerous fields in estimating the survival of patients who had undergone breast cancer (Yigit, 2013), detecting the presence of hypothyroidism and hyperthyroidism (Aversano et al., 2021), predicting the site of e-coli bacteria proteins and yeast (Chumuang, 2018), predicting diabetes in patients (Abedini et al., 2020), predicting whether a mail is a spam or not, image segmentation and image identification, identifying the type of glass based on the oxide content, and pattern recognition like identifying the type of iris flowers based on their characteristics (Yigit, 2013).

The factors that affect the accuracy of classifiers are

- Overlapping between different classes
- Class imbalance
- Length of the class boundary in MST of the dataset (N1 complexity metric)
- Presence of noise
- Presence of a non-linear boundary between the classes
- Existence of multiple chunks of the classes that are distributed variably throughout the dataset

This work refers to the above factors as data complexity. This work divides the original problem into subproblems. These subproblems have reduced data complexity compared to the original problem. Classification models are designed for each subproblem, and a dynamic ensemble is designed using these models.

Ensemble learning is an approach where diverse models are combined to yield better predictive performance by reducing the error due to bias and variance. Some prominent ensembling techniques include bagging, boosting, and stacking (Breiman, 1996; Garcia et al., 2021; Schapire, 2013). Zhou et al. (2010) proposed methods that dynamically choose a subset of component classifiers of the ensemble to get the prediction for a given test instance. DES(Dynamic Ensemble Selection) is a prominent method that determines x sets of classifiers for prediction from the X set of classifiers (x<X). Some of the renowned DES models include DES-KNN (Kinal & Woźniak, 2020), META-DES (Cruz et al., 2015), K-Nearest Oracles Union (KNORA-U) (Ko et al., 2008), K-Nearest Oracles Eliminate (KNORA-E) (Cavalin & Suen, 2013), EKSL (Bektaş, 2022), and DES for Fault Classification (DESFC) (Zheng et al., 2022).

Some ensembling methods divide the dataset with different combinations with repetitions to produce a multi-set structure, while few techniques iteratively adjust the dataset. Bagging involves dividing the original dataset into k datasets using the bootstrap method, which samples the data uniformly with replacement. Each dataset is utilized for training the classifier model M_i, and the result is determined by calculating the majority voting of prediction results by all the models M_i, i = 1…N (Breiman, 1996). Random forest utilizes two selection procedures to reduce variance. Bootstrap or random subspace selection divides problems into subproblems and constructs decision trees for each dataset (Breiman, 2001). Easy Ensemble and Balanced cascade algorithms create subproblems by under-sampling the majority class instances to create balanced subproblems. These methods focus on

* Correspondence to: Maulana Azad National Institute of Technology, Bhopal, Madhya Pradesh 462007, India.
E-mail addresses: sendtosow@gmail.com (Sowkarthika B.), manasi_gyanchandani@yahoo.co.in (M. Gyanchandani), wadhvani_rajesh@rediffmail.com (R. Wadhvani), sanyamshukla@gmail.com (S. Shukla).

https://doi.org/10.1016/j.eswa.2022.119437
Received 9 August 2022; Received in revised form 30 October 2022; Accepted 10 December 2022
Available online 19 December 2022

reducing the data complexity caused by the class imbalance (Liu et al., 2009).

An ideal ensemble should consist of accurate and diverse component classifiers (Yang, 2011). Component classifiers will be accurate if the subproblems have less data complexity. The classifiers are diverse when the subproblems' training instances are different. Although the problem is divided into subproblem structures, the split is random in Bagging, and Random forest algorithms (Breiman, 1996, 2001). The Easy Ensemble and Balance Cascade use random undersampling to create balanced subproblems. It should be noted that these approaches do not consider the data distribution for the creation of subproblems (Liu et al., 2009). The proposed methodology generates subproblems considering data complexity. The subproblem creation of the proposed method is illustrated in Fig. 4. Each class in a subproblem is dichotomous. Any non-linear problem is divided into a set of linear problems. Due to this, the subproblems created are simpler and more diverse. The subproblem's size is much smaller than the other ensemble models with a reduced length of decision boundary for each subproblem (Smith & Jain, 1988).

The final prediction of the ensemble can be obtained by deploying different approaches like majority voting, weighted voting, meta classifiers, and dynamic selection of component classifiers. Bagging, Easy ensemble, and Random forest use majority voting. The proposed study assigns weights to each component classifier based on the distance of the test instance to the separating plane of each subproblem. The high weights are assigned to the close instances from hyperplanes. The proposed dynamic ensemble approach geometrically identifies the weightage of results yielded by component classification models to be used for the given test instance, leading to enhancement of the accuracy of the model.

This work is organized as follows. The motivation and novelty of the proposed model is briefed in Section 2. Section 3 gives an overview of the data complexity measures utilized in the proposed work. Methodologies that consider data complexity for solving classification problems are also discussed in Section 3. Section 4 describes the proposed methodology. The next section elaborates on the synthetic data generation and the parameter settings used for the study. The results obtained and their analysis is given in Section 6. The last section concludes the work.

2. Motivation and novelty of the proposed model

The real world is full of classification problems. Classification finds its application in Image recognition: Fingerprint recognition, Face recognition, Histopathological, Traffic control systems, Action detection, Disease detection with medical imaging, object identification in satellite images, etc. Signal Classification: ECG signal classification, EEG signal classification, Ball bearing fault detection, Rebar Detection.Text Classification: Spam Filtering, Topic Labelling, Sentiment analysis, Marketing, etc.

Several classification algorithms are developed for classifying the data. Different classifiers perform differently over different datasets (Jain et al., 2000). Researchers are continuously developing novel classifiers and examining their wide applications. Data complexity is regarded as a crucial aspect in the development of the proposed work.

Along with the classifiers, researchers have also proposed ensemble-based approaches, which reduce the error due to variance and error due to bias. EKSL (Bektaş, 2022) and DESFC (Zheng et al., 2022) are some of the recent ensemble based methods. The majority of the ensemble approaches used today, such as bagging, boosting, and stacking, Breiman (1996), Garcia et al. (2021), Schapire (2013) do not divide the original problem down into smaller problems. Easy ensemble Balance cascade (Liu et al., 2009) divides the original imbalanced classification problem into balanced subproblems. These methods create subproblems with reduced data complexity in terms of class imbalance. This work targets developing an ensemble method with lower data complexity in terms of the number of concepts. The contribution of the proposed work are as follows:

- Novel data complexity-based approach for the creation of subproblems.
- Novel approach to combine the outcomes of component classifiers. Utilizing a weighted ensembling technique, weights assigned to the component classifiers' predictions are based on the dataset characteristics. The weights assigned to each classifier's result are inversely proportional to the distance between the test data and the hyperplane generated for the corresponding classifier. Since the hyperplane for SVM-RBF is in feature space, the length of the test data from the hyperplane is calculated as the minimum distance between the test data and the support vectors that are distributed around the hyperplane.

3. Related work

3.1. Data complexity and its metrics

Predictions in classification models are highly dependent on the dataset characteristics (Cano, 2013; Ho, 2008; Kim, 2010). Since the input data plays a significant role, the characteristics of the data can be analyzed with the help of data complexity measures which elucidate the behavior of the dataset. The data complexity metrics utilized in this paper are briefed in the following subsections. The ideal values of the complexity measures for better classifier performance should be as presented in Table 1.

3.1.1. Fisher's discriminant ratio (F1)

For each feature, this measure computes the distance between the mean of classes and their spread to understand the separability of the classes in every dimension (Eq. (1)). The mean and variance of instances in each class are computed for each feature and examined. The maximum value of f_i indicates that the distance between the mean of the classes is high for the feature 'i'. i.e., the classes can be clearly discriminated, while a low value implicates that there is overlap among the classes. The feature f_i efficiently distinguishes samples belonging to both classes for higher values of f_i (Ho & Basu, 2002). For $i = 1 \ldots d$ dimensions, F1 is defined as

$$F1 = MAX(f_i),\ where\ f_i = \frac{(\mu_1 - \mu_2)^2}{\sigma_1^2 + \sigma_2^2} \tag{1}$$

3.1.2. Volume of overlap region (F2)

This computes the overlap region of all the class instances. For each class, the start and endpoints of overlap are estimated for a particular dimension, and the volume of the overlap region is calculated by the product of overlap in all dimensions. The overlap is huge for the large region extracted by Eq. (2), which is observed with high values of F2. If the overlap is minimum or non-existent, then the value of F2 is low, implying that the classes are easily separable, and the prediction accuracy is high. F2 = 0 if there is no overlap in at least one of the feature dimensions, indicating that the feature is efficient in differentiating classes. For c_1, c_2 = class and f_i = feature, F2 is given by

$$F2 = \pi_i \frac{MIN(max(f_i, c_1), max(f_i, c_2)) - MAX(min(f_i, c_1), min(f_i, c_2))}{MAX(max(f_i, c_1), max(f_i, c_2)) - MIN(min(f_i, c_1), min(f_i, c_2))} \tag{2}$$

3.1.3. Feature efficiency (F3)

This complexity measure describes the contribution of the individual feature in separating two classes eluding any overlap. The fraction of number of instances that are not in the overlapping region to the total number of instances yields F3 (Eq. (3)). High values suggest that a significant number of instances are separable by classes, minimizing overlap. F3 $\approx$ 1 implies that the problem is separable (Ho & Baird, 1998).

$$Feature\ Efficiency\ F3 = \frac{Instances\ separable\ in\ a\ region}{Total\ Points} \tag{3}$$

Table 1
Ideal values of data complexity metrics for accurate classification.

Data complexity metrics	Values of measures to be optimal	Value range
F1	High	[0, ∞]
F2	Low	[0, 1]
F3	High	[0, 1]
N1	Low	[0, 1]

3.1.4. Fraction of points on class boundary(N1)

The class boundary is estimated by constructing Minimum Spanning Tree and counting the number of instances in edges connecting two different classes (referred as length of class boundary in this work). N1 value is defined by the ratio of the length of the class boundary to the total number of points in the dataset. N1 offers a better understanding on the pattern of data distribution. If the value is high, it suggests that too many points lie on the class boundary, making the problem complex and intractable (Smith & Jain, 1988).

3.2. Existing methodologies to reduce data complexity

3.2.1. Methodologies that handle class overlapping and noise

Few classifier models were developed with consideration of data complexity. CANB (Classification Algorithm based on Naïve Bayes) (Xiong et al., 2013), two Ellipsoid SVM algorithm (Czarnecki & Tabor, 2014), One-Vs-One methodology (Saez et al., 2019) were modeled to handle the overlapping between classes in a dataset.

Filtering techniques like preprocessing the dataset is the most commonly used technique to remove noisy instances. The cross-validation filter is utilized to eliminate noise where the dataset is divided into n parts and $n-1$ parts are used to create $n-1$ models and the nth part is given as input to the model where noise is identified. This procedure is repeated for all the n parts (Mousavi & Langston, 2017). Boosting is incompetent with noise since the weights of the noisy instances are increased in boosting process, considering them weak learners (Verbaeten & Van Assche, 2003).

3.2.2. Methodologies that handle class imbalance and non-linearity

Some models were designed to handle class imbalance. Oversampling of minority class instances and downsampling of majority class instances are done to avoid the class imbalance problem. SMOTE (Chawla et al., 2002), Borderline-SMOTE (Han et al., 2005), Safe-level SMOTE (Bunkhumpornpat et al., 2009), Tomek-link (Tomek, 1976), variants of ELM algorithm like WELM (Weighted ELM (Zong et al., 2013)), Boosting WELM (Li et al., 2014), class-specific ELM (Raghuwanshi & Shukla, 2018), minimum class variance class-Specific ELM (Raghuwanshi & Shukla, 2021), variances-constrained ELM (Liu et al., 2020), and class-specific cost regulation ELM (Xiao et al., 2017) are also used in imbalanced learning.

Some ensembling techniques are designed to solve imbalanced data. RUSBagging algorithm performs bagging over the ensemble of models where bootstrap training samples are distributed in a balanced way (Wallace et al., 2011). SMOTEBagging combines the resampling techniques like over-sampling, under-sampling, and SMOTE (Wang & Yao, 2009). RUSBoost undersamples the majority class instances before Boosting is performed (Seiffert et al., 2010), whereas SMOTEBoost creates synthetic instances of minority class and uses the AdaBoosting technique (Chawla et al., 2002). RUSBoosting performs better since the number of instances in SMOTEBoosting is huge, causing high time consumption for the SMOTEBoosting model to learn (Vuttipittayamongkol et al., 2021). GRSOM technique balances by growing new data on the ring (Chetchotsak et al., 2015). UBKELM model utilizes random undersampling with bagging where Kernelized ELM is used as a classifier for handling imbalanced datasets (Raghuwanshi & Shukla, 2019).

For the non-linear distribution of data, the radial basis function kernel is utilized where the input space is converted to a higher dimensional feature space to fit the dataset so that the classes can be distinguished. Prominent classification algorithms like k-NN (Ougiaroglou et al., 2007), CART (Quinlan, 1986), neural networks (Jielai et al., 2015), Naïve Bayes (Schneider, 2003) also handle non-linearity. Ensembling techniques like Random Forest, and AdaBoost are used in non-linear classification.

3.3. Ensembling models for classification

Two recently proposed Ensemble-based techniques are utilized in this work to analyze the performance of the proposed model and are briefed in the following subsections.

3.3.1. EKSL

EKSL is a dynamic ensemble model where the imbalanced datasets are balanced by dividing the majority dataset into k-datasets by the k-Means clustering algorithm and merging with the minority dataset to yield k-datasets. SVM classifiers are constructed over these datasets. Binomial Logistic Regression is utilized in ensembling the classifiers. Bektaş (2022).

3.3.2. Dynamic ensemble selection model for fault detection and classification (DESFC)

Random forest with decision trees is the ensembling approach utilized here. Euclidean distance between each test data instance and Validation set is estimated, k-NN validation samples are chosen, and a diversity matrix is constructed for each classifier based on the chosen validation instances. Hierarchical clustering is performed by using each classifier as an individual cluster and the diversity matrix as the distance matrix. Finally, the classifier with the best performance in each cluster is identified, the weighted vector is derived through the accuracy of the classifiers chosen in the cluster, and the final predictions are made (Zheng et al., 2022).

4. Proposed ensemble-based solution

The proposed model (see Fig. 1) considers the dataset as a single problem and partitions the problem into subproblems in such a way that the complexity metrics of each subproblem are minimal. This proposed classifier ensemble creates subproblems with reduced data complexity in terms of

- Reduced length of the class boundary in N1
- Reduced volume of the overlapping region
- Improved feature efficiency
- Increased Separability of the classes (minimum or no overlapping)
- Ability to handle class imbalance problem

4.1. Design of component classifiers

The dataset is taken as input, and the euclidean distance between all the instances is computed, and the adjacency matrix A is constructed. $A_{i,j}$ = Euclidean distance between instances i and j, where $A_{i,j} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ for 2-dimensional instances $i(x_1, y_1)$ and $j(x_2, y_2)$.

Minimum Spanning Tree is constructed based on the values from the adjacency matrix. Prim's algorithm is used for building MST from all the instances in the dataset. Prim's and Kruskal's algorithms are the prominent greedy algorithms used in constructing MST. Prim's is chosen here since the run time complexity of this algorithm is $O(E + VlogV)$, which is smaller than $O(ElogV)$ of Kruskal's. Dense graphs give better results for Prim's. In our case, $E \gg V$, which is $E = O(V^2)$, the complexity of Kruskal's will be substantially higher than Prim's. Prim's algorithm starts with an empty spanning tree and maintains two

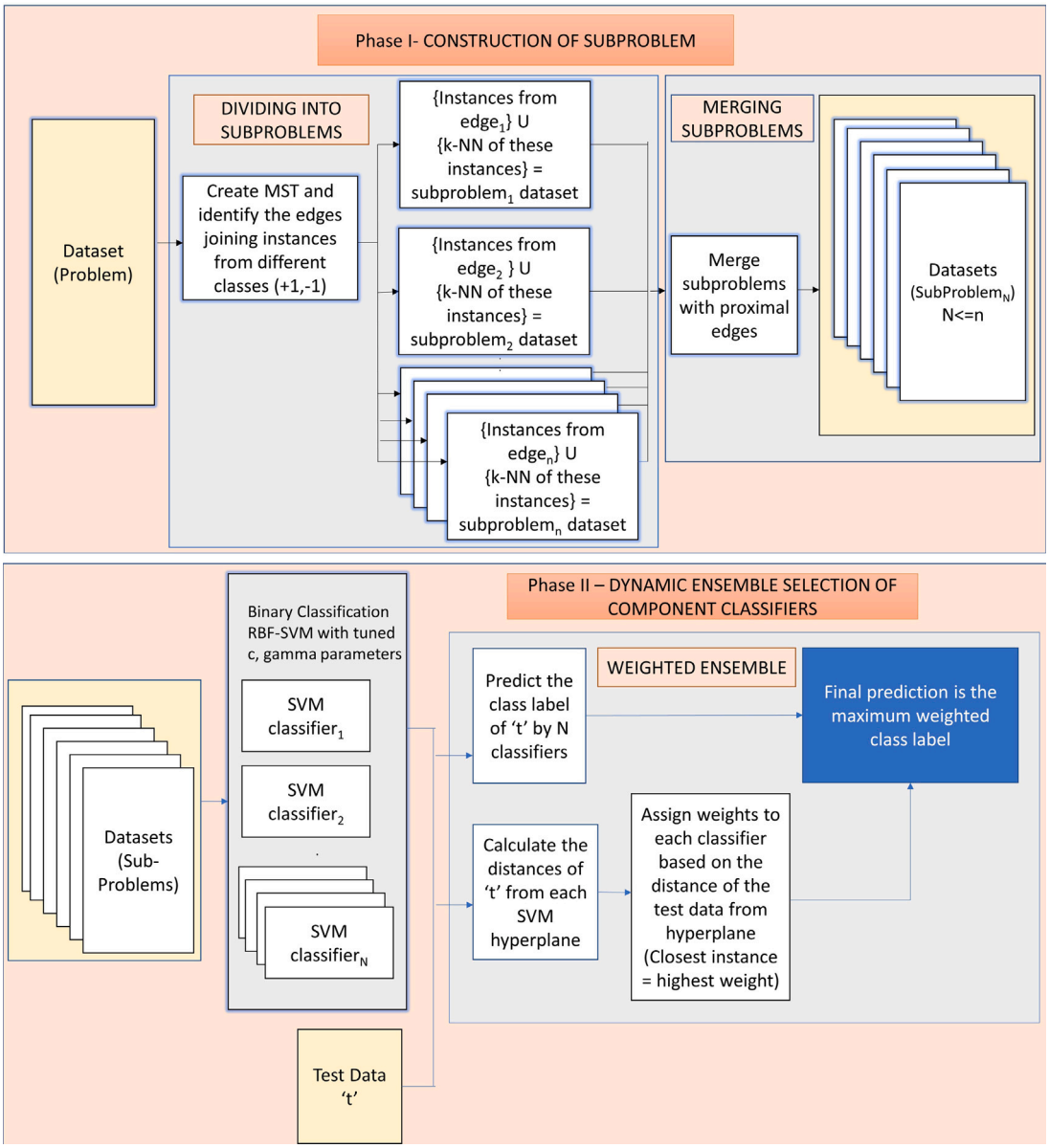

Fig. 1. Proposed model.

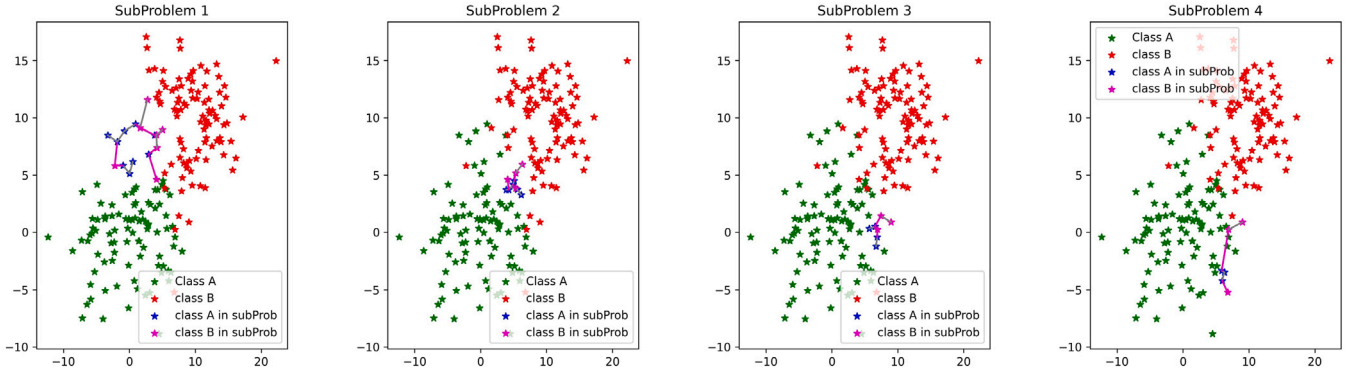

Fig. 2. Subproblem structure for the dataset (4 subproblems are generated for the given dataset. Each highlighted edge contains instances that constitute a subproblem).

sets of vertices where one set retains information on the vertices that MST covers and the other set includes vertices that are not in the first set. The minimum weighted edge connecting these two vertices sets is included in every step (Cormen et al., 2001).

Algorithm 1: Decomposing a problem into subproblems.

Input: Dataset (Problem)
Output: SubProblem set
for *each dataset* ***do***
1. *Create a 5-fold cross-validation of the dataset*
for *each fold* ***do***
2. *Calculate Adjacency Matrix*
3. *Create Minimum Spanning Tree (MST) by Prim's algorithm*
4. *Estimate the instances on the edges connecting different classes in MST*
5. *Calculate 2-NN of the instances from the previous step*
6. $Subproblem_i$=$InstancesOnEdge_i \cup 2\text{-}NN(InstancesOnEdge_i)$
7. $SubproblemSet$=$SubproblemSet \cup Subproblem_i$
while *Last and Current subProblem Sets are different* ***do***
for *each subproblem i,j* ***do***
if $NoOfInstances(subproblem_i(classA) \cap subproblem_j(classA)) > 0$ *and* $NoOfInstances(subproblem_i(classB) \cap subproblem_j(classB)) > 0$ ***then***
(Merging Subproblems i and j if they are closer)
8. $SubProblem_i = SubProblem_i \cup SubProblem_j$
9. $RemoveDuplicateInstances(Subproblem_i)$
10. $SubproblemSet = SubproblemSet - SubProblem_j$
end if
end for
end while
end for
end for

After constructing MST, we need to parse the tree to discover the edges joining two different classes. The instances of interest are the ones that lie proximal to the decision boundary, and they can be estimated with the help of these edges. The 2-NN of each instance on the decision boundary is found and added to the subproblem. The nearby instances' position reveals the spread and shape of the class boundary. The 2-NN are elected in such a way that each instance from a class has two neighbors that belong to the same class. Instances at either end of the edge and their 2-NN constitute a subproblem. Initially, there are six instances for any subproblem before merging. Dividing the problem structure into subproblems is represented in Fig. 2. The colors exemplify the class instances, where the edges connect the adjacent instances of different classes. The number of subproblems will be high for a dataset with huge overlapping between classes. In addition, the number of subproblems will be low for readily differentiable class instances.

Sometimes, two edges can be adjacent, and hence, they belong to the same decision boundary. These edges can be merged into the same subproblem. Two edges can be merged if and only if they are close i.e., the instances of the subproblems in both the classes of binary classification overlap. In Fig. 3, it could be seen that the two edges are proximal, which is estimated by the overlap of instances in both classes. So, these are merged into a single subproblem.

After constructing the subproblem, RBF-SVM (Radial Basis Function -Support Vector Machine) is utilized in constructing component classifiers with optimally tuned C and $gamma$ parameters. The hyperplanes obtained after tuning C and gamma are depicted in Fig. 4. Later, the ensembling of all these component classifiers is done.

4.2. Dynamic selection of component classifier

Each RBF-SVM tuned classifier is utilized, and a dynamic ensemble model is constructed by assigning appropriate weights to each classifier. The detailed approach is given in algorithm 2.

The test data is taken as input, and the minimum distance of each test data to the hyperplane separating the classes is calculated. Weights

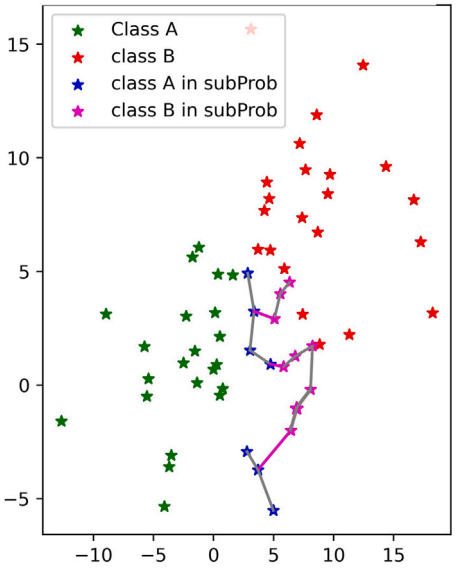

Fig. 3. Merging subproblems (a part of the subproblem structure where adjacent edges are merged. The 3 edges highlighted in magenta constitute 3 subproblems are merged).

Algorithm 2: Dynamic Ensembling Model.

Input: Subproblems that contain instances from different classes
Output: Dynamic Ensemble Selection of component classifiers
for *each* $Subproblem_i$ *in* $SubproblemSet$ ***do***
1. *Train instances in* $Subproblem_i$, *i=1...N using RBF-SVM with tuned C and gamma values where the accuracy is maximum (N=number of subproblems), to build model* M_i.
2. *Calculate the minimum distance of the test data from the hyperplane separating the classes for each model* M_i. ***(In case of non-linear multi-dimensional hyperplane, the minimum distance of the test data with the closest support vector is calculated for each model*** M_i***.)***
3. *Assign weights to each model* M_i *based on the distance calculated (Min. Distance = Max. weight).*
4. *Predict the class value for the test data 't' by* M_i, *i=1...n.*
5. *Estimate the final predicted value for the test data 't' using (6).*
end for

are assigned to each test data t in model i using Eq. (4) where $i = 1 \ldots N$ and N = number of models generated.

$$Weight_{i,t} = 1 - \frac{MinDist_{i,t}}{\sum_t Dist} \tag{4}$$

where $Weight_{i,t}$ = Weight of the class value predicted by Model M_i for the test instance t and $MinDist_{i,t}$ = Minimum distance between the test instance t and Model M_i, and $\sum_t Dist$ = Sum of all the minimum distances from the test instance t to every Model M_i.

The class value generated by any model M_i will have more weight in predicting the target class when the test data is proximal from the hyperplane of M_i. The estimated weight takes value from [0,1] for ease of computation and analysis. The weight of a test data t for a model m is normalized by dividing the minimum distance of t from $hyperplane_m$ by the sum of the distances of t from $hyperplane_i$ where $i = 1..N$ and the result is subtracted from 1, which in turn makes the small distances gain large weights and the large distances to procure smaller weights.

$$Weight_{i,t} \alpha \frac{1}{Distance_{i,t}} \tag{5}$$

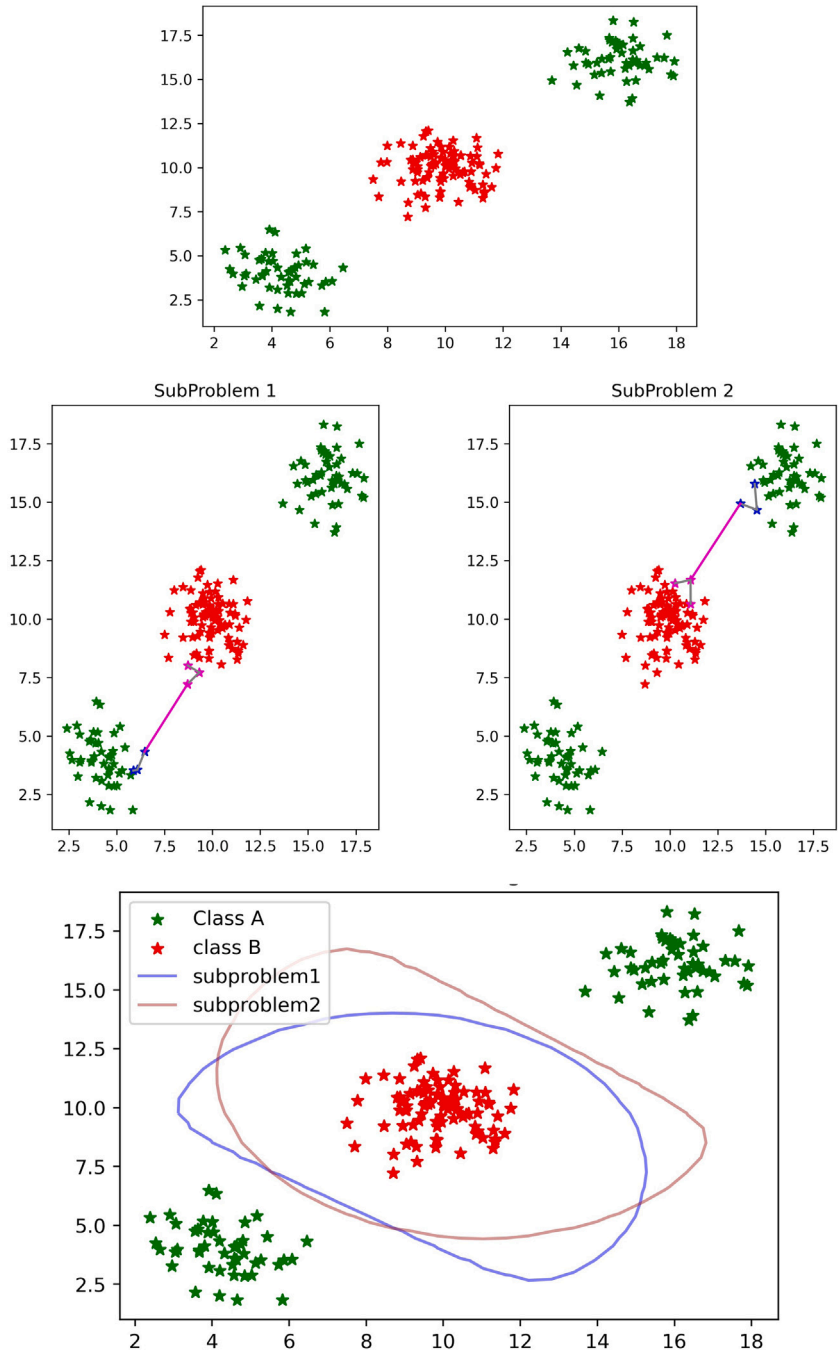

Fig. 4. Row 1- Original dataset and Dataset with MST.
Row 2- Subproblem structure.
Row 3- SVM Hyperplanes generated for each subproblem set (2 subproblems = 2 hyperplanes).

After calculating the distances, the class values of test data t with each classifier i (of model M_i) are predicted. For a test data t, the weights calculated for every class value are added separately, and the target class value is estimated as the class label with the highest weights as shown in Eq. (6).

$$ClassValue(t) = \left\{ c \mid Max(\sum_{c \in C1,C2} Weight_{i,t}) \right\} \quad (6)$$

where $i = 1..N$, t = test instance, and $C1, C2$ are the binary classes in the dataset. Later, the G-mean of the proposed model is compared with the existing prominent models to analyze the performance of a given dataset.

5. Synthetic data generation and parameter settings

5.1. Synthetic data generation

The proposed model has to be evaluated for different distributions, varying degrees of overlapping across classes, different shapes of decision boundary, multiple disjuncts where more than one concept per class are present. The mean(μ) and standard deviation(σ) utilized for the Gaussian distribution in the generation of synthetic datasets in our study are summarized in Table 2 and different distribution of instances are illustrated in Fig. 5. Synthetic datasets generated for binary classification avails two features for better comprehension and

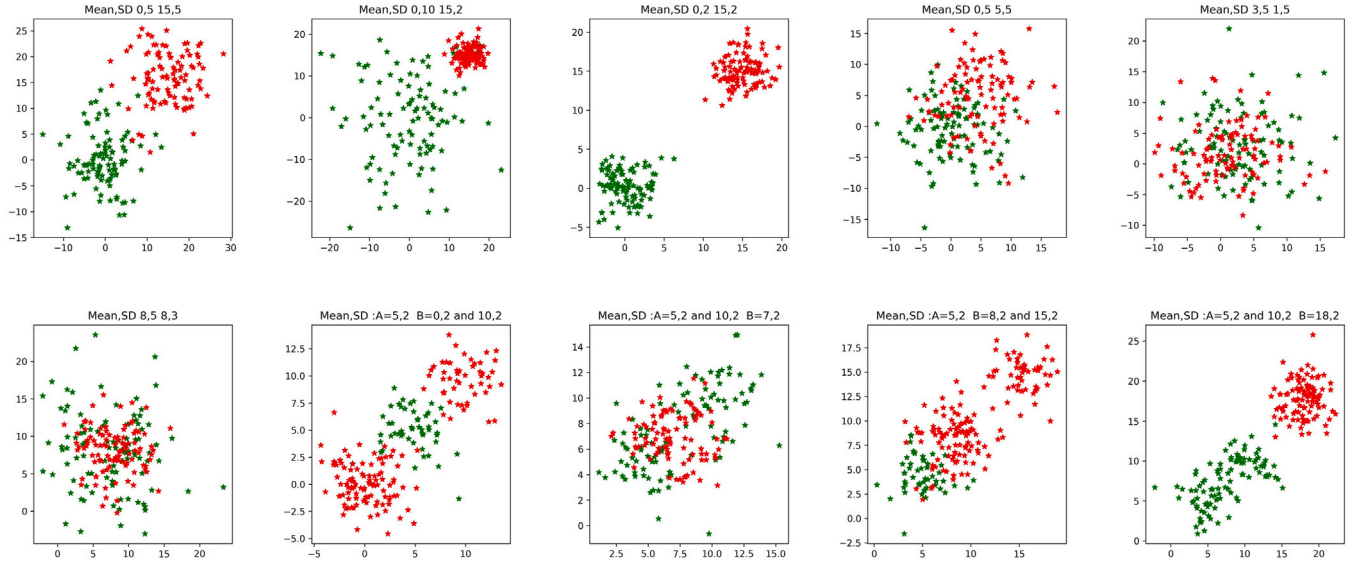

Fig. 5. Instance Distribution in different synthetic datasets considered for the study (Last four subfigures show the presence of more than one concept).

Table 2
Mean (μ) and standard deviation (σ) for the Gaussian distribution of synthetic datasets.

S.No	μ, σ of class A	μ, σ of class B
1	0, 5	15, 5
2	0, 10	15, 2
3	0, 2	15, 2
4	0, 5	5, 5
5	3, 5	1, 5
6	8, 3	8, 5
7	5, 2	0, 2 10, 2
8	5, 2 10, 2	7, 2
9	5, 2	8, 2 15, 2
10	5, 2 10, 2	18, 2

simpler graphical representation and visualization of the data. The dataset size is maintained constant at 200.

5.2. Parameter settings

The proposed model is additionally evaluated using 28 binary classification datasets extracted from the KEEL dataset repository (Alcalá-Fdez et al., 2011).

This work presents the average results obtained using 5-fold cross-validation. The optimal values of performance are quoted by tuning C and $gamma$ where the range of these parameters are $C = \{2^{-15}, 2^{-13}, 2^{-11} \ldots 2^{18}, 2^{20}\}$ and $gamma = \{2^{-15}, 2^{-13}, 2^{-11} \ldots 2^{18}, 2^{20}\}$. C is a regularization parameter used to establish the point of tolerance of the misclassification of points and generalization, and $gamma$ is the kernel width and suggests the influence of each training example. Choosing a fitting $(C, gamma)$ pair is critical in deciding the accuracy of the model and reflecting the complex nature of the data. Although tuning different C and $gamma$ parameters for every subproblem would be optimal for improving the precision of the proposed model, it is computationally very intensive. So, the optimal C and $gamma$ are tuned for a problem, and the same parameters are used for every subproblem in all folds in 5-fold cross-validation. The same $(C, gamma)$ pair is used for RBF-SVM, base classifiers in Bagging, and DESFC for a fair evaluation of results.

5.3. Performance evaluation metrics

The G-mean is utilized to evaluate the performance of the classification algorithm. It is calculated using Eq. (7) where TP = True Positive, FN = False Negative, TN = True Negative, and FP = False Positive.

$$G - mean = \sqrt{TPR \cdot TNR} \tag{7}$$

$$\begin{aligned} TruePositiveRate(TPR) &= \frac{TP}{TP + FN} \; and \\ TrueNegativeRate(TNR) &= \frac{TN}{TN + FP} \end{aligned} \tag{8}$$

6. Experiment, results, and their analysis

In this section, the results presented are analyzed after performing the experiments. Table 3 compares the data complexity metrics (F1, F2, F3, N1) of the synthetic dataset with the decomposed subproblems, while Table 4 presents the comparative difference between the dataset and subproblems for five real-world datasets from the KEEL repository. The results are analyzed in Section 6.1.

The fraction of points on the class boundary does not effectively represent N1 because the N1 value should reduce when the data differentiability increases. The current formula yields the same N1 metric for a tiny dataset with minimum overlap and a large dataset with medium overlap. N1 metric fails to represent different complexities with appropriate N1 metric values. The enormous datasets get minimal N1 values since the number of points on the class boundary is divided by the dataset size yields a small value. So, the formulation for N1 has been changed to accommodate the appropriate N1 value according to the level of overlapping given in Eq. (9). The modified value of N1 ranges from [0, N] where N=number of instances in the dataset.

$$N1 = Length\ of\ the\ class\ boundary \tag{9}$$

6.1. Results and analysis of data complexity measures

Tables 3 and 4 aid in analyzing data complexity measures using synthetic datasets and the real-time dataset from the KEEL repository respectively. It can be seen that the values yielded by the subproblems have reduced data complexity compared to the entire dataset. Figs. 6 and 7 provide the data complexity metrics for different instance distributions. The subfigures in both figures provide the complexity metric comparison between the dataset and the subproblem, which can

Table 3
Data complexity measures for synthetic dataset for distribution from Table 2 and its subproblems.

S.No	No. of Subprob	Problem (Dataset)				Subproblems			
		F1	F2	F3	N1	F1	F2	F3	N1
1	1	**6.194**	0.109	0.959	1	Min: 4.077 Max: 4.077	**0**	Min: **1.0** Max: 1.0	Min: 1 Max: **1**
2	2	3.139	0.482	0.515	2	Min: **11.21** Max: 13.00	**0**	Min: **1.0** Max: 1.0	Min: 1 Max: **1**
3	1	34.715	0	1.0	1	Min: **1226.65** Max: 1226.65	**0**	Min: **1.0** Max: 1.0	Min: 1 Max: **1**
4	6	1.165	0.533	0.201	28	Min: **1.66** Max: 11.23	**0.12**	Min: **0.57** Max: 1.0	Min: 1 Max: **10**
5	18	0.252	0.714	0.121	60	Min: **1.75** Max: 12.39	**0**	Min: **0.33** Max: 1.0	Min: 1 Max: **15**
6	16	0.041	0.470	0.216	59	Min: **0.207** Max: 12.197	**0.015**	Min: **0.56** Max: 1.0	Min: 1 Max: **10**
7	4	0.010	0.440	0.476	6	Min: **0.496** Max: 7.56	**0**	Min: **0.87** Max: 1.0	Min: 1 Max: **4**
8	9	0.016	0.465	0.289	34	Min: **0.114** Max: 13.383	**0**	Min: **0.56** Max: 1.0	Min: 1 Max: **11**
9	5	3.10	0.065	**0.889**	11	Min: **5.50** Max: 14.05	**0**	Min: 0.8 Max: 1.0	Min: 1 Max: **5**
10	1	8.82	0.013	1.0	1	Min: **24.57** Max: 24.57	**0**	Min: **1.0** Max: 1.0	Min: 1 Max: **1**

Table 4
Data complexity measures for KEEL repository dataset and its subproblems Inst-Number of Instances and Attr- Number of Attributes in the dataset.

Dataset	Inst	Attr	No. of subprob	Problem (Dataset)				Subproblems			
				F1	F2	F3	N1	F1	F2	F3	N1
Banana	5299	2	125	0.012	0.772	0.009	292	Min: **0.052** Max: 72.25	**0**	Min: **0.571** Max: 1.0	Min: 1 Max: **5**
Haberman-2	306	3	31	0.208	0.717	0.555	75	Min: **1.022** Max: 20.25	Min: 0 Max: **0.415**	Min: **0.814** Max: 1.0	Min : 1 Max: **6**
Haberman-3	306	3	23	0.215	0.617	0.587	81	Min: **1.013** Max: 10	Min: 0 Max: **0.476**	Min: **0.815** Max: 1.0	Min: 1 Max: **21**
bupa-3	345	6	30	0.054	0.099	0.193	111	Min: **0.53** Max: 49.99	Min: **0** Max: **0.015**	Min: **0.694** Max: 1.0	Min: 1 Max: **14**
Monk-2–5	432	7	7	1.055	0.666	1.0	103	Min: **1.787** Max: 23.93	**0**	**1.0**	Min: 1 Max: **45**

be explained as — Subfigure 1 in the figure depicts the original data distribution with two different classes highlighted in different colors and illustrates the MST for all the instances, while subfigure 2 gives the MST for the subproblems with edges belonging to the same subproblem in the identical color. Subfigure 3 gives the volume of the overlapping region for the entire dataset, while subfigure 4 gives the volume of the overlapping region in the subproblem. Subfigure 5 highlights instances in the overlapping region of the dataset that assists in calculating F3, while subfigure 6 highlights instances in the overlapping region of the subproblem. Subfigure 7 brings out the distance between the mean of the binary classes, while subfigure 8 brings out the distances between the mean of the classes belonging to a different subproblem.

The following analysis can be made from the results obtained. As presented in Tables 3 and 4, the values of complexity metrics for a dataset with differentiable classes almost remain the same between the problem and the subproblem. It can be inferred from cases 1,2, 3, and 10 in Table 3. In these circumstances, the complexity metrics of subproblems are almost identical to the original problem. The overlapping of class instances escalates the complexity. Overlapping in the dataset causes a fall in F1 and F3, and a rise in F2 and N1 values, while the subproblems exhibit values with reduced data complexity (high F1 and F3 values, low F2 and N1 values). Fig. 7 shows that subproblems of Multi-concept datasets yield better complexity values than the original dataset.

6.2. Experimentation on the G-mean index of the model

The following algorithms are the prominent models used for classification. Accuracy rates and G-mean index of these algorithms are compared with the proposed model for a better understanding of the model.

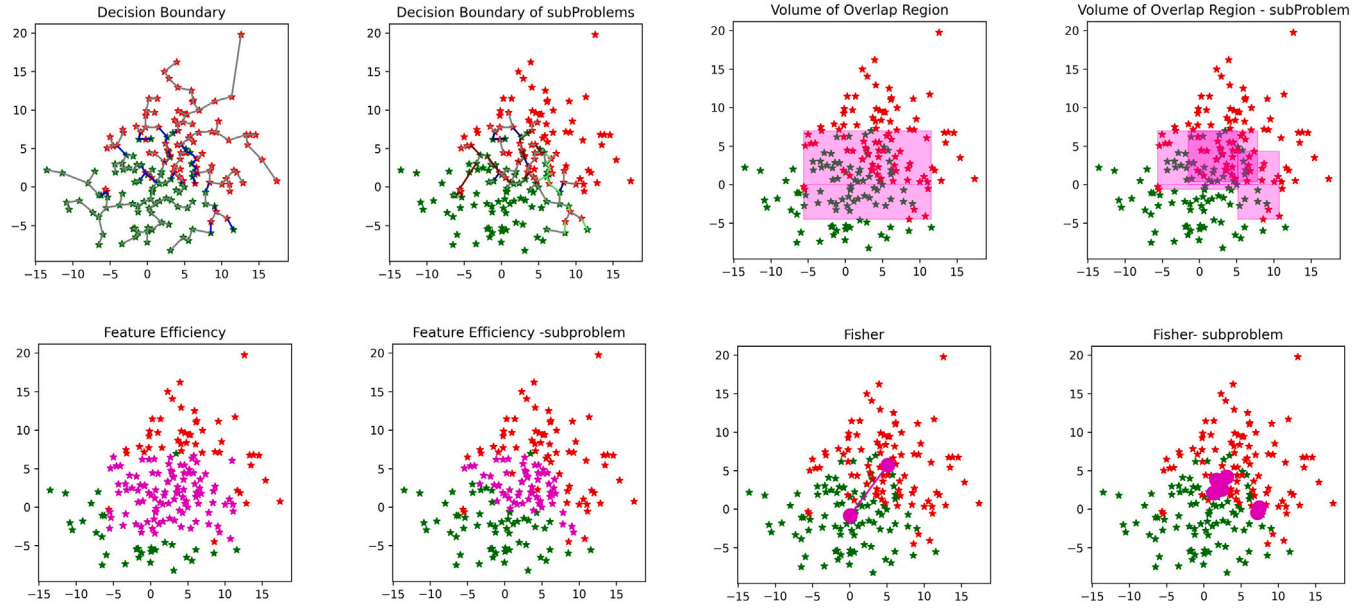

Fig. 6. Data complexity metrics for overlapping dataset
(1. Original dataset with MST. 2. MST with edges of the same subproblem are in the identical color. 3. Volume of Overlapping region for the entire dataset. 4. Volume of Overlapping region in the subproblem set. 5. Instances for calculating F3 in the dataset. 6. Instances in F3 for the subproblem set. 7. Distance between the mean(F1) of the classes in the dataset. 8. Distance between the mean of the classes in subproblem set).

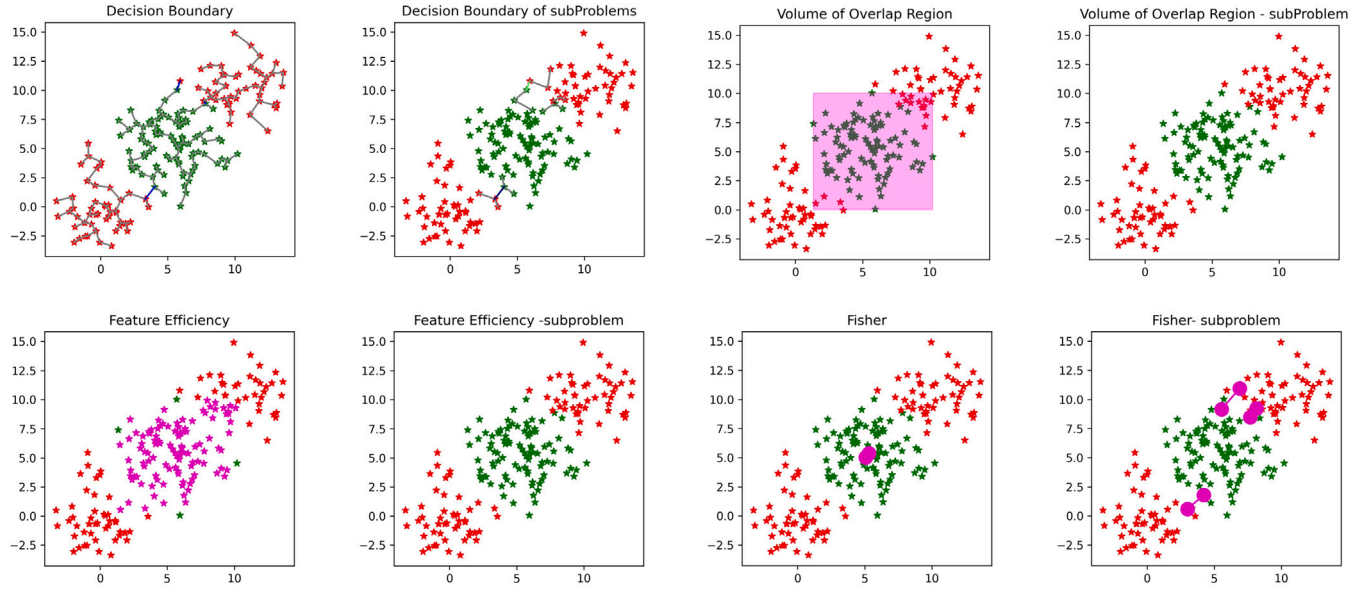

Fig. 7. Data complexity metrics for the dataset with multiple clusters in the same class.

Linear-SVM: This algorithm works better with separable datasets. Overfitting is low. This model needs tuning C and gamma parameters to get the desired accuracy (Cortes & Vapnik, 1995; Meyer et al., 2003).

RBF-SVM: This algorithm deals with non-linear data. The hyperplane is constructed in $k+1$-dimension for a k-dimensional dataset.

k-NN: It is a lazy learning algorithm where the test data is assigned a value based on the majority value of its k neighbors. It does not need training time. New data can be added anytime without any impact on the accuracy. The Nearest Neighbor algorithm is simple, efficient and easy to implement (Ougiaroglou et al., 2007). 3-NN is used in this study.

Naïve Bayes Multinominal: This is the probabilistic classification algorithm based on the Bayes theorem. It is simple, fast, and works best with natural language processing (NLP) (Schneider, 2003).

Decision Tree algorithm: Simple and easy to comprehend and visualize. This algorithm develops a decision tree where features are assigned as internal nodes, branches are the decision taken, and the leaves are the class values the data belongs to Quinlan (1986).

Bagging: Each dataset from bootstrap is utilized for training the classifier model M_i. The test data is given to the model, and the result is determined by calculating the majority of prediction results by model M_i, i = 1…N. In this study, Bagging utilizes RBF-SVM as the classifier with pre-determined C and gamma parameters obtained from the grid search, which is the same as the existing model (Breiman, 1996).

Random Forest: It is an ensembling technique where decision trees are utilized as base models and bootstrap yields datasets to be trained for each component classifier. Majority vote of the base models yields the final predicted value for any test data. Ho (1995)

Table 5
Average Accuracy and G-mean for 5-fold cross-validation of the synthetic datasets SVM-Support Vector Machine, KNN-k-Nearest Neighbor, NB-Naive Bayes, DT-Decision Tree, Prop-Proposed Model.

No	No. of Subprob	Accuracy of the Models					G-mean of the Models				
		SVM	KNN	NB	DT	Prop	SVM	KNN	NB	DT	Prop
1	3	97	97	97	**98**	**98**	96.998	96.998	96.998	97.976	**97.9904**
2	3	97.5	**98.5**	**98.5**	**98.5**	**98.5**	97.536	98.486	**98.516**	98.3784	**98.516**
3	1	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**
4	23	73	73.5	72.6	65.5	**78.1**	72.788	71.982	71.776	64.886	**76.5032**
5	17	58	58.5	62	52	**68.5**	57.431	56.692	61.727	50.734	**67.589**
6	16	59.5	65.5	64.5	56.5	**69**	59.234	65.0262	63.9722	54.672	**67.5652**
7	16	61.5	68	64	62.5	**76**	59.689	67.6082	62.07	62.4	**75.3696**
8	19	49.5	68	69	66.5	**73.8**	48.542	67.72	65.57	63.62	**73.022**
9	11	**73.1**	68	68.5	69.5	72	**71.081**	67.11	67.916	69.43	70.9704
10	1	**100**	**100**	**100**	99	**100**	**100**	**100**	**100**	98.92	**100**

Table 6
Statistical t-test results of accuracy and G-Mean calculated for 5 folds with $\alpha = 0.10$.

Comparison (With proposed model)	Accuracy				G-mean			
	tstat	p	Dof	h(0.1)	tstat	p	Dof	h(0.1)
Linear-SVM	−2.487903373	0.017270313	9	Rejected	−2.444987818	0.018530089	9	Rejected
k-NN	−3.309435526	0.004546044	9	Rejected	−3.096844939	0.006393699	9	Rejected
Naïve Bayes	−3.143264719	0.005932746	9	Rejected	−2.920252776	0.008512409	9	Rejected
Decision Tree	−3.14257595	0.005939329	9	Rejected	−3.142609894	0.005939004	9	Rejected

Gradient Boosting: This is a sequential learning ensembling technique where the wrongly predicted results are assigned higher weights in the next level. Optimization of arbitrary differentiable loss functions is performed with this algorithm. Friedman (2001)

EKSL: Here, the author has estimated the optimal value of the parameters in experimentation to be k > 3, C = 50, and these values are used in performing the experiments (Bektaş, 2022).

Dynamic ensemble selection model for fault detection and classification (DESFC): SVM is employed instead of decision trees as a base classifier for a fair comparison. Bootstrap aggregation is used, and SVM models are built on the datasets. Later Euclidean distance between each test data instance and Validation set is estimated, and the procedure in 3.3.2 is performed to acquire the results (Zheng et al., 2022).

The average accuracy and G-mean values of 5 folds of the synthetic dataset are presented in Table 5.

6.3. Result and analysis of accuracy and the G-mean index of the model

As it can be seen from Table 5, the G-mean of the proposed model remains almost the same as the existing models when the data is linearly separable. These classes can be readily distinguished. High accuracy and G-mean index are guaranteed for this type of problem. All the existing classification models work better in this scenario, as seen in the table with cases 1,3, and 10. The proposed model performs better than the existing models while analyzing complex patterns in the dataset. In this case, the value of F1 and F3 will be low, and F2 and N1 will be high for the entire dataset, implying that there is overlap between data in different dimensions. In these scenarios, the proposed model decomposes the dataset into subproblems, with each subproblem having high F1, F3, and low F2, N1 values. It can be seen from the table that the overlapping data has the best accuracy and G-mean rate with the proposed model.

Also, it can be seen from cases 7,8,9 and 10 of Table 5 that the multiple disjuncts of the same class have better accuracy with the proposed model. The same class has more than one concept (different mean and SD for the same class in a dataset), resulting in instances being distributed as clusters in the dataset. The estimation of class labels on this kind of data can be inaccurate, which can be eluded with the help of the proposed model. So, the proposed model outperforms the existing models in overlapping cases and cases with multiple concepts.

For further evaluation, a pairwise t-test is performed on the accuracy rate and the G-mean value of the existing algorithms and the proposed algorithm from the 5-fold cross-validation method of the synthetic dataset to understand the statistical significance of the dynamic model proposed where significance level $\alpha = 0.10$. The mean difference between the observed pair has to be non-zero. The smaller p-value suggests that there is a significant difference between pairs. P-value indicates the likelihood of observing the null hypothesis, suggesting that the mean difference between the observed pairs is zero. Table 6 presents that the p-values are found to be low, which rejects the null hypothesis (suggesting the mean difference is non-zero) with the confidence of 90% and suggests that the proposed methodology is relevant and significant.

In the case of multi-dimensional data, the study has estimated the distance of the test data in the Ensemble section from the support vectors of the model rather than the hyperplane. The hyperplane of the component RBF-SVM classifier is in the k+1 dimension for the k feature dataset. The hyperplane has to be converted to input space since the test data is k-dimensional rather than k+1-dimension. Since identifying a definite position for cutting the hyperplane to make it k-dimensional based on the projection of the nearest distance from k-dimensional test data was not possible, the distance is estimated from the support vectors. The support vectors are chosen since it gives the approximate spread of the hyperplane. The minimum distance between test data and the closest support vector to the test data is chosen for further calculation of weights. Predictions are done by component classifiers and the G-mean is calculated.

28 real-time multi-dimensional datasets from the KEEL dataset repository have been utilized for the study. Table 7 lists C and gamma values yielded by the proposed model, and compares the G-mean values obtained with other prominent models. It could be seen that the proposed model has approximately the same G-mean value as the other models and sometimes, the proposed model outperforms the other existing models.

The subproblems created here are simpler and diverse, which leads to the development of component classifiers that are accurate and diverse. An ensemble with accurate and diverse component classifiers gives better performance. Moreover, the proposed work presents a novel method for integrating the component classifiers' outcomes, improving performance. Therefore, the proposed model achieves better results which can be perceived from Table 7.

The datasets utilized in the model from the KEEL repository are less than 30. So pairwise t-testing is performed. The G-Mean of each model is compared separately with the proposed model in a pairwise manner

Table 7
Average G-mean of different models for 5- fold cross-validation on KEEL repository datasets with optimal C and Gamma values. RF-Random Forest, GB-Gradient Boosting.

Dataset	C	GAMMA	G-mean										
			Linear SVM	RBF SVM	k-NN	Naïve Bayes	Decision Tree	Bagging	RF	GB	DESFC	EKSL	Proposed model
Haberman	1	0.00195	4.79	42.82	48.20	44.39	51.46	51.10	49.66	46.45	32.45	38.12	**65.12**
Bupa	128	0.00195	64.37	59.98	61.48	58.35	61.64	61.57	65.96	66.25	63.01	47.26	**66.50**
Appendicitis	2	1	56.13	57.51	56.96	74.71	56.89	71.65	60.14	58.73	**76.02**	66.99	72.36
Pima	3.051e−05	3.051e−05	70.21	61.25	66.05	**71.98**	66.98	66.38	71.71	68.40	51.92	70.02	69.54
Sonar	2048	0.00195	74.86	75.96	**79.64**	68.87	70.47	78.57	78.62	74.99	76.31	45.38	66.61
SpectfHeart	32	3.051e−05	66.89	59.85	53.21	75.59	56.95	63.76	30.42	52.69	51.97	15.93	**75.75**
Ecoli-0_vs_1	2	4	96.98	**98.97**	97.61	93.98	97.93	**98.97**	**98.97**	97.34	98.68	60.78	95.65
Ecoli-1	2	4	82.11	85.52	83.03	71.46	82.73	84.28	83.37	82.50	83.12	48.56	**86.60**
Ecoli-2	32	0.125	70.07	80.56	**90.61**	42.48	83.75	90.51	83.94	87.01	80.94	60.10	83.35
Ecoli-3	3.051e−05	16	89.57	89.57	86.60	67.25	86.0	86.90	68.78	69.41	89.57	29.72	**89.83**
Glass-0-1-2-3_ vs_4-5-6	2	0.5	87.99	93.46	88.61	85.82	88.22	81.72	**93.61**	91.43	86.70	66.51	91.57
Glass-0	524288	0.0078125	64.52	75.49	72.76	66.98	79.88	71.34	81.36	**88.25**	70.35	63.56	75.79
Glass-1	3.051e−05	0.125	15.38	38.76	**76.86**	60.36	66.39	51.82	68.51	68.69	43.64	35.22	69.19
Glass-2	1	256	92.01	92.01	88.29	42.58	85.02	92.01	92.01	20.05	60.63	61.65	**93.08**
Glass-3	3.051e−05	0.125	93.44	93.09	92.97	86.38	90.65	**93.90**	90.57	92.01	**93.90**	23.38	**93.90**
Glass-6	1	0.125	88.86	84.52	85.47	87.47	86.98	83.38	89.40	89.50	87.64	36.76	**90.20**
Glass-0-1-6_vs-2	3.051e−05	0.5	**91.1**	**91.1**	86.96	34.95	84.87	21.54	25.47	**91.10**	**91.10**	67.67	89.33
Shuttle-6_vs_2–3	3.051e−05	3.051e−05	**95.65**	**95.65**	**95.65**	95.21	**95.65**	**95.65**	**95.65**	**95.65**	**95.65**	43.47	**95.65**
Shuttle-c2_vs_c4	3.051e−05	3.051e−05	**95.65**	**95.65**	**95.65**	95.21	**95.65**	**95.65**	**95.65**	**95.65**	**95.65**	43.47	**95.65**
Iris	3.051e−05	3.051e−05	**100**	92	**100**	**100**	**100**	**100**	**100**	**100**	81	96.81	**100**
New-thyroid-1	128	0.00195	97.59	97.96	89.12	**98.48**	94.10	60.03	96.14	95.84	95.60	65	94.50
New-thyroid-2	2048	0.00012	97.09	96.31	89.13	98.86	92.10	56.87	96.14	94.80	**98.95**	73	94.18
Wisconsin-2	3.051e−05	3.051e−05	96.39	86.53	**96.87**	96.64	92.90	96.28	96.59	96.57	96.23	96.28	92.90
Yeast-1	3.051e−05	4	41.37	35.76	62.84	20.64	64.28	53.23	66.27	64.12	55.23	30.59	**74.25**
Yeast3	3.051e−05	1	50.24	52.65	81.98	45.88	82.79	83.05	83.22	82.83	46.73	20.92	**90.26**
Yeast-0-5-6-7-9_VS_4	3.051e−05	3.051e−05	90.34	90.34	87.69	34.10	82.19	90.34	50.36	62.95	90.34	37.30	**91.09**
Yeast-1_vs_7	3.051e−05	3.051e−05	**93.44**	**93.44**	91.90	18.37	86.65	**93.44**	38.77	48.28	**93.44**	47.50	86.65
Yeast-2_vs_4	3.051e−05	3.051e−05	**90.07**	**90.07**	89.48	28.75	86.96	89.48	89.40	83.06	**90.07**	45.31	89.48

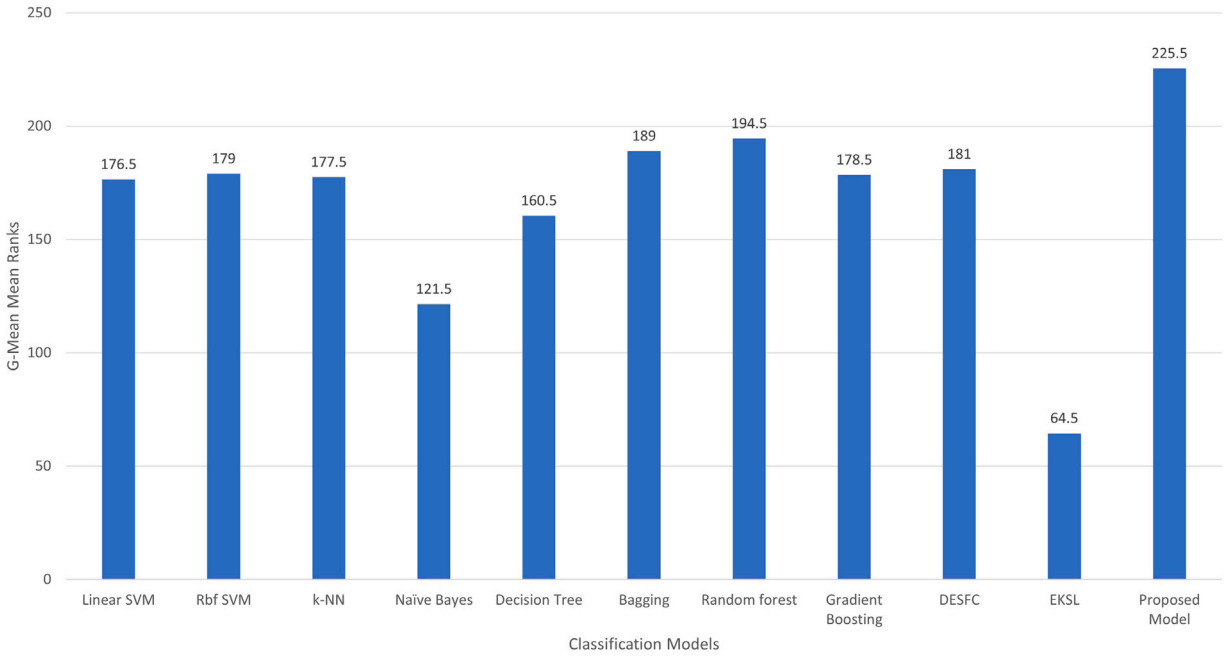

Fig. 8. Friedman's mean ranks of G-mean for the KEEL dataset.

with respect to each dataset. To illustrate the difference between the G-Means of the models, the paired t-test is conducted under the following three presumptions (LLC, 2022).

1. Datasets utilized are independent. Each dataset is different from other dataset.

2. Each of the paired measurements is obtained from the same dataset. Each dataset is evaluated for both models in consideration.

3. The measured G-Mean differences between each model are normally distributed (Normality can be assessed with the Shapiro–Wilk test, which can sometimes be misinterpreted based on the sample size. Moderate skewness is tolerated in unimodal sample distribution. Non-parametric methods can be utilized, or transforming the data to another scale (logarithmic functions, for instance) can be done before analyzing the normality) (Pandis, 2015; Watson et al., 2021). In our work, common logarithm (log10) of the G-Mean differences is estimated, and Shapiro–Wilk test utilizes this data to analyze the normality. The Shapiro–Wilk tests did not reject the null hypothesis, which assumes that the data is normally distributed since the probability values generated for all the models > 0.5 for $\alpha = 0.05$.

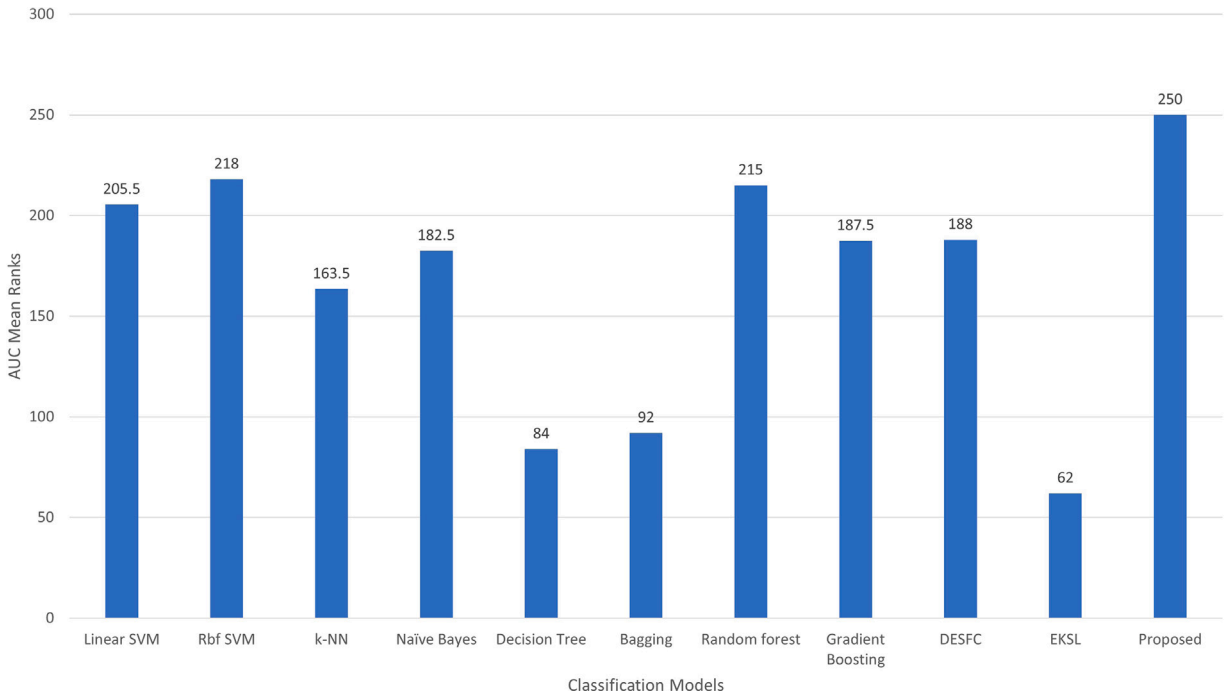

Fig. 9. Friedman's mean ranks of AUC for the KEEL dataset.

Table 8
Statistical t-test results of G-Mean calculated for KEEL repository datasets with $\alpha = 0.05$.

Comparison (With proposed model)	G-mean			
	t-stat	p	Dof	h(0.050)
Linear-SVM	−2.39196	0.011993	27	Rejected
RBF-SVM	−2.621352	0.007106	27	Rejected
k-NN	−2.10875	0.022192	27	Rejected
Naïve Bayes	−4.01319	0.000214	27	Rejected
Decision Tree	−3.72826	0.000452	27	Rejected
Bagging	−2.446348	0.010614	27	Rejected
Random Forest	−2.48576	0.009707	27	Rejected
Gradient Boosting	−2.5906	0.007631	27	Rejected
DESFC	−2.509349	0.009199	27	Rejected
EKSL	−8.662571	1.41E−09	27	Rejected

Table 9
Anova test for G-Mean values of KEEL repository datasets with $\alpha = 0.05$.

Classification models	Count	Sum	Average	Variance
Linear-SVM	28	2157.11	77.03964	612.4064
RBF-SVM	28	2206.78	78.81357	378.7545
k-NN	28	2295.62	81.98643	205.3385
Naïve Bayes	28	1865.74	66.63357	665.9325
Decision Tree	28	2270.08	81.07429	186.731
Bagging	28	2232.98	79.74929	250.6134
Random Forest	28	2078.85	74.24464	578.2479
Gradient Boosting	28	2149.5	76.7678	364.1730
DESFC	28	2208.22	78.865	370.8731
EKSL	28	1437.26	51.33071	414.563
Proposed Model	28	2378.98	84.96357	117.1154

Source of variation	SS	df	MS	F	*P*-value	F crit
Between groups	24396.44304	10	2439.6443	6.474	4.81E−09	1.8626
Within groups	111908.21	297	376.7953			
Total	136304.658	307				

t-test and ANOVA (Analysis of Variance) test with significance level $\alpha = 0.05$, and Friedman's Mean Rank Test are performed on the data from Table 7 and are presented in Tables 8 and 9 and Fig. 8 respectively. The null hypothesis is rejected (inferred from a very small *p*-value), indicating a significant difference between the sample sets drawn from the two models, which means that there is a significant difference between the proposed and existing models.

The models' performance has also been additionally analyzed using an AUC table. AUC measure gives a detailed analysis of the models' prediction with probabilities. The proposed model assigns weights to each class value generated by the component classifier in ensembling to achieve precise results. These weights are further utilized for analysis of the AUC. The AUC probabilistic framework helps to understand that the proposed model distinguishes the classes efficiently and not randomly since the AUC values from the Table 10 for the proposed model are far higher. It is evident from the Table 10 that the proposed model identifies and proficiently categorizes the classes. Later pairwise t-tests and Friedman's Mean Rank Test are also performed on the results to exhibit the significance of the proposed model (presented in Table 11 and Fig. 9).

The ROC (Receiver Operating Characteristics) curve is a visualization tool that aids in analyzing the model that differentiates classes. This curve presents the performance of a model at different classification thresholds. True Positive Rate(TPR) and False Positive Rate(FPR) are calculated for a model based on the predicted and actual value and plotted as a graph. The probability of the predicted class is gathered for every test data and ranked, and the graph is plotted based on these values. Fig. 10 presents the ROC curve for different models (Linear and RBF-SVM, k-NN, Naïve Bayes, Decision Tree model, Bagging, Random Forest, Gradient Boosting, DESFC, EKSL, and the proposed model) on the data from the KEEL repository. It can be observed from the results that the proposed model differentiates the classes better than other models.

$$TPR = \frac{TP}{TP + FN} \quad and \quad FPR = \frac{FP}{FP + TN} \tag{10}$$

6.4. Analysis of computational cost

The computational cost of the proposed model is dataset-specific. The time complexity is nearly the same as basic RBF-SVM for a separable dataset since few subproblems are generated, and the subproblem size is meager compared to the dataset size. The time complexity of the

Table 10
AUC of different models for 5- fold cross-validation on KEEL repository datasets with optimal C and Gamma values.

Dataset	C	GAMMA	AUC										
			Linear SVM	RBF SVM	k-NN	Naïve Bayes	Decision Tree	Bagging	RF	GB	DESFC	EKSL	Proposed model
Haberman	32	0.0019	0.61	0.68	0.63	0.68	0.54	0.61	0.65	0.61	0.53	0.56	**0.71**
Bupa	128	0.00012	0.72	0.75	0.65	0.64	0.63	0.71	**0.77**	0.72	**0.77**	0.63	**0.77**
Appendicitis	32768	0.0078	**0.86**	**0.86**	0.77	0.78	0.64	0.77	0.81	0.81	**0.86**	0.75	**0.86**
Pima	3.051e−05	3.051e−05	**0.83**	0.79	0.73	0.81	0.67	0.73	0.78	0.77	0.69	**0.83**	0.79
Sonar	3.051e−05	0.00012	0.72	0.69	0.65	0.64	0.60	0.71	**0.77**	0.72	**0.77**	0.48	0.70
SpectfHeart	32	0.00012	0.78	0.83	0.69	**0.85**	0.62	0.71	0.78	0.75	0.62	0.41	**0.85**
Ecoli-0Vs1	3.051e−05	4	**0.99**	**0.99**	0.98	**0.99**	0.97	**0.99**	0.95	0.98	**0.99**	0.92	**0.99**
Ecoli1	128	1	**0.94**	0.93	0.90	0.90	0.82	0.90	0.93	0.93	0.78	0.66	0.91
Ecoli2	8	0.5	0.94	**0.96**	0.94	0.94	0.84	0.94	**0.96**	0.94	0.78	0.71	**0.96**
Ecoli3	0.0078125	16	**0.93**	0.91	0.87	0.90	0.77	0.86	0.90	0.89	0.90	0.83	**0.93**
Glass0123vs456	3.051e−05	0.03125	0.96	0.97	**0.98**	0.96	0.87	0.5	**0.98**	**0.98**	0.96	0.78	**0.98**
Glass0	8	0.5	0.83	0.87	0.85	0.77	0.76	0.5	**0.89**	0.88	**0.89**	0.81	**0.89**
Glass1	32	0.125	0.63	0.81	0.86	0.66	0.71	0.5	0.87	**0.89**	0.66	0.58	0.78
Glass2	128	1	0.51	0.71	0.77	0.73	0.41	0.5	0.70	0.66	0.70	0.48	**0.87**
Glass3	2	0.125	0.92	0.95	0.97	0.77	0.62	0.5	0.80	0.77	0.92	0.48	**0.98**
Glass6	3.051e−05	0.125	**1.0**	**1.0**	0.99	**1.0**	0.81	0.5	**1.0**	0.96	0.98	0.69	**1.0**
Glass016vs2	3.051e−05	0.125	0.95	0.97	**0.99**	0.96	0.98	0.5	**0.99**	0.98	0.98	0.85	0.97
Shuttle6vs23	3.051e−05	3.051e−05	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	0.83	**1.0**
Shuttlec2vsc4	3.051e−05	3.051e−05	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	0.5	**1.0**	**1.0**	**1.0**	0.77	**1.0**
Iris	3.051e−05	3.051e−05	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	**1.0**	0.99	**1.0**
Newthyroid1	2048	3.051e−05	**1.0**	**1.0**	0.98	**1.0**	0.94	0.74	**1.0**	0.99	**1.0**	0.74	**1.0**
Newthyroid2	8192	3.051e−05	0.99	0.99	0.99	**1.0**	0.91	0.69	0.99	0.99	0.99	0.83	**1.0**
Wiscosin2	3.051e−05	3.051e−05	**0.99**	**0.99**	0.98	0.98	0.92	0.97	**0.99**	**0.99**	**0.99**	**0.99**	**0.99**
Yeast1	3.051e−05	0.000122	0.78	0.72	0.71	0.77	0.66	0.66	0.75	0.75	0.75	0.65	**0.83**
Yeast3	3.051e−05	0.125	0.96	**0.97**	0.91	0.95	0.82	0.87	0.88	0.93	**0.97**	0.83	**0.97**
Yeast05679vs4	8	4	0.84	0.81	0.81	0.80	0.72	0.70	**0.88**	0.82	0.85	0.72	**0.88**
Yeast1vs7	32	1	0.81	0.76	0.70	0.80	0.62	0.59	0.75	0.79	**0.83**	0.55	0.79
Yeast2vs4	32	1	0.92	0.97	0.88	0.90	0.83	0.87	**0.98**	0.95	0.92	0.69	0.97

Table 11
Statistical t-test results of AUC calculated for KEEL repository datasets with $\alpha = 0.05$.

Comparison (With proposed model)	AUC			
	t-stat	p	Dof	h(0.050)
Linear-SVM	−2.3938	0.01194	27	Rejected
RBF-SVM	−2.4081	0.011567	27	Rejected
k-NN	−4.33916	8.98E−05	27	Rejected
Naïve Bayes	−3.949	0.000253	27	Rejected
Decision Tree	−6.7308	1.53E−07	27	Rejected
Bagging	−5.6563	2.63E−06	27	Rejected
Random Forest	−2.13224	0.02117	27	Rejected
Gradient Boosting	−2.695	0.00598	27	Rejected
DESFC	−3.2678	0.001475	27	Rejected
EKSL	−8.13353	4.88E−09	27	Rejected

Table 12
Computational Cost of different classification models.

Classification model	Training time	Prediction time
Linear-SVM	$O(n^3)$	$O(d)$
RBF-SVM	$O(n^3 + n^2 d)$	$O(n_{sv} d)$
k-NN	–	$O(n^d)$
Naïve Bayes	$O(n^d)$	$O(d)$
Decision Tree	$O(n^2 d)$	$O(d)$
Bagging	$O(n_{est}(n^3 + n^2 d))$	$O(n_{est}(n_{sv} d))$
Random Forest	$O(n^2 d n_{tree})$	$O(n_{tree} d)$
Gradient Boosting	$O(n^2 d n_{tree})$	$O(n_{tree} d)$
DESFC	$O(n_{est}(n^3 + n^2 d))$	$O(n_{sv} d + val^2 + k val^2)$
EKSL	$O(n^2 + n_{est}(n^3 + n^2 d) + nd)$	$O(n_{sv} d + (d+1)n)$
Proposed Model	$O(n^2 + nlogn + n_{sub}(n_k^3 + n_k^2 d))$	$O(n_{sv} d + n_{sub} n_{sv} d)$

Table 13
Computational Cost of the proposed Model.

Activity	Computation	Computational cost
Training	Adjacency Matrix construction	$O(n^2)$
	Construction of MST	$O(nlogn)$
	Construction of Component Classifier(RBF-SVM)	$O(n_{sub}(n_k^3 + n_k^2 d))$
Prediction	Prediction by component classifiers	$O(n_{sv} d)$
	Ensembling	$O(n_{sub} n_{sv} d)$

proposed model for any complex dataset is higher than Random Forest and Gradient Boosting. But, the problem of generating an optimal decision tree for a large dataset is an NP-Complete problem (Avellaneda, 2019). Therefore, RF and GB are not suitable for datasets of large size.

The proposed method is an ensemble of RBF-SVM. So the proposed method is compared with three other RBF-SVM-based ensemble methods, namely DESFC, EKSL, and Bagging. The computational complexity of RBF-SVM is $O(n^3 + n^2 d)$ where n=number of instances in a dataset. Therefore, the minimal computational cost of the algorithm with an ensemble of RBF-SVMs (Bagging, DESFC, and EKSL) will be $O(n_{est}(n^3 + n^2 d))$. This value is higher than $O(n^2 + nlogn + n_{sub}(n^3 + n^2 d))$ since a single quadratic problem generated with one component classifier in the existing RBF-SVM ensemble models is higher than the quadratic problems generated with the proposed model. Although the number of subproblems varies with the dataset, the size of the subproblem is meager in the proposed model.

For an SVM-RBF component classifier, the computation cost in other ensemble models is $O(n^3)$, and the proposed model is $O(n_k^3)$, where n_k = the maximum number of instances in all the subproblems. Here $n_k \ll n$ (the size of any subproblem is minimal compared to the dataset). Therefore, the proposed model's complexity is low compared to Bagging, DESFC, and EKSL.

For n = Number of instances in a dataset, d = Number of features in the dataset, n_{sv} = Number of support vectors, val = size of validation set chosen, k = number of clusters utilized, n_{tree} = Number of trees generated, n_{est} = number of estimators in an ensemble, n_k = maximum number of instances in all the subproblems, and n_{sub} = number of subproblems generated, the computational cost of different models are given by Table 12 and the proposed model is explained in Table 13.

7. Conclusion and future work

This work proposes a novel ensemble-based technique with a novel approach for decomposing the original problem into subproblems. SVM is used for developing classification models for these subproblems.

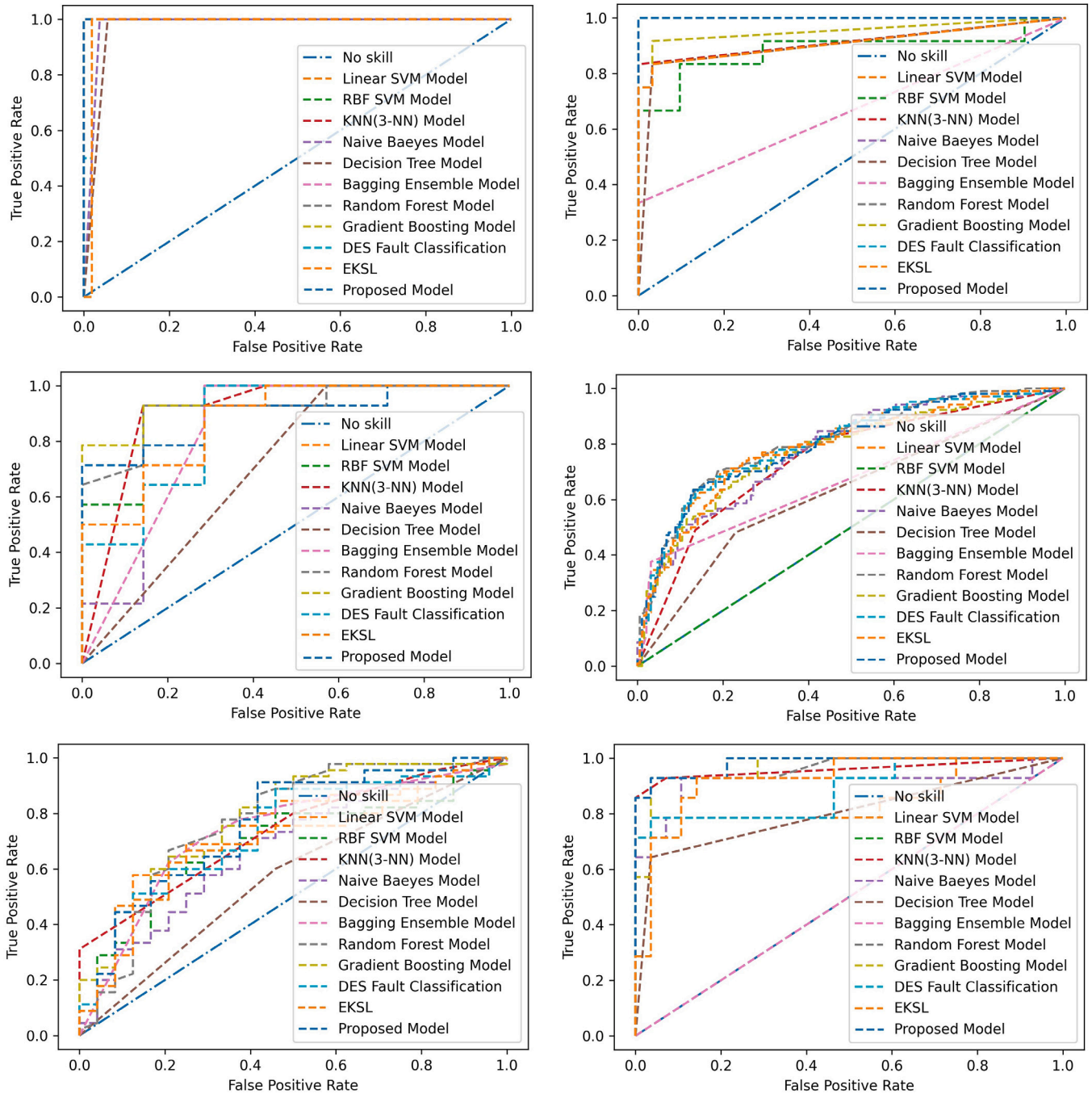

Fig. 10. AUC-ROC curve of different models for datasets Ecoli-0-1-3-7_vs_2-6, New-thyroid1, Appendicitis, Yeast-1, Bupa and Glass-0-1-2-3_vs_4-5-6 (from left to right, top to bottom).

The outcomes of these models are combined using a novel dynamic ensemble algorithm. The proposed work is compared with the state-of-art classifiers and two recently proposed dynamic ensemble-based techniques. It is to be noted that the subproblems have lower data complexity than the original problem. The proposed work is exhaustively evaluated using synthetic datasets and benchmark datasets taken from the KEEL repository. The performance in terms of G-Mean is computed for the classifiers under consideration. It has been observed that the proposed model outperforms other techniques. This has been further validated by the statistical tests conducted. The future work includes the extension of the proposed approach to multi-class classification. MST is utilized in our proposed model to identify the concepts present in a dataset. Other heuristic techniques can be employed to explore the concepts of a dataset. A study on identifying the optimal k-value in the construction of a subproblem can be done (The k-value refers to the k-nearest neighbor instances added to each class in the subproblem).

CRediT authorship contribution statement

Sowkarthika B.: Conceptualization and design of study, Experimentation and coding, Drafing the manuscript. **Manasi Gyanchandani:** Conceptualization and design of study, Review and analysis of the manuscript. **Rajesh Wadhvani:** Conceptualization and design of study, Review and analysis of the manuscript. **Sanyam Shukla:** Conceptualization and design of study, Review and analysis of the manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Abedini, M., Bijari, A., & Banirostam, T. (2020). Classification of Pima Indian Diabetes Dataset using Ensemble of Decision Tree, Logistic Regression and Neural Network. *Ijarcce*, *9*(7), 1–4. http://dx.doi.org/10.17148/ijarcce.2020.9701.

Alcalá-Fdez, J., Fernández, A., Luengo, J., Derrac, J., García, S., Sánchez, L., & Herrera, F. (2011). KEEL data-mining software tool: Data set repository, integration of algorithms and experimental analysis framework. *Journal of Multiple-Valued Logic and Soft Computing*, *17*(2–3), 255–287.

Avellaneda, F. (2019). Learning Optimal Decision Trees from Large Datasets Florent Avellaneda.

Aversano, L., Bernardi, M. L., Cimitile, M., Iammarino, M., Macchia, P. E., Nettore, I. C., & Verdone, C. (2021). Thyroid disease treatment prediction with machine learning approaches. *Procedia Computer Science*, *192*, 1031–1040. http://dx.doi.org/10.1016/j.procs.2021.08.106, URL: http://dx.doi.org/10.1016/j.procs.2021.08.106.

Bektaş, J. (2022). EKSL: An effective novel dynamic ensemble model for unbalanced datasets based on LR and SVM hyperplane-distances. *Information Sciences*, *597*, 182–192. http://dx.doi.org/10.1016/j.ins.2022.03.042.

Breiman, L. (1996). Bagging predictors. *Machine Learning*, *24*(2), 123–140. http://dx.doi.org/10.1007/bf00058655.

Breiman, L. (2001). Random forests. *Machine Learning*, *45*(1), 5–32.

Bunkhumpornpat, C., Sinapiromsaran, K., & Lursinsap, C. (2009). Safe-level-SMOTE: Safe-level-synthetic minority over-sampling technique for handling the class imbalanced problem. *5476 LNAI*, In *Advances in knowledge discovery and data mining* (pp. 475–482). Berlin, Heidelberg: Springer Berlin Heidelberg, http://dx.doi.org/10.1007/978-3-642-01307-2_43.

Cano, J. R. (2013). Analysis of data complexity measures for classification. *Expert Systems with Applications*, *40*(12), 4820–4831. http://dx.doi.org/10.1016/j.eswa.2013.02.025.

Cavalin, P., & Suen, C. (2013). Dynamic selection approaches for multiple classifier systems. *Neural Computing and Applications*, *22*, http://dx.doi.org/10.1007/s00521-011-0737-9.

Chawla, N. V., Lazarevic, A., Hall, L. O., & Bowyer, K. W. (2002). SMOTEBoost: Improving prediction of the minority class in boosting. *Journal of Artificial Intelligence Research*, *16*, 321–357. http://dx.doi.org/10.1007/978-3-540-39804-2_12.

Chetchotsak, D., Pattanapairoj, S., & Arnonkijpanich, B. (2015). Integrating new data balancing technique with committee networks for imbalanced data: GRSOM approach. *Cognitive Neurodynamics*, *9*(6), 627–638. http://dx.doi.org/10.1007/s11571-015-9350-4.

Chumuang, N. (2018). Comparative Algorithm for Predicting the Protein Localization Sites with Yeast Dataset. In *Proceedings - 14th international conference on signal image technology and internet based systems, SITIS 2018* (pp. 369–374). IEEE, http://dx.doi.org/10.1109/SITIS.2018.00064.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2001). Introduction to algorithms, (2nd). The MIT Press.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Leaming*, *20*, 273–297. http://dx.doi.org/10.1007/BF00994018, URL: http://dx.doi.org/10.1007/BF00994018.

Cruz, R. M., Sabourin, R., Cavalcanti, G. D., & Ing Ren, T. (2015). META-DES: A dynamic ensemble selection framework using meta-learning. *Pattern Recognition*, *48*(5), 1925–1935. http://dx.doi.org/10.1016/j.patcog.2014.12.003, URL: https://www.sciencedirect.com/science/article/pii/S0031320314004919.

Czarnecki, W. M., & Tabor, J. (2014). Two ellipsoid Support Vector Machines. *Expert Systems with Applications*, *41*(18), 8211–8224. http://dx.doi.org/10.1016/j.eswa.2014.07.015, URL: http://dx.doi.org/10.1016/j.eswa.2014.07.015.

Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, *29*, 1189–1232.

Garcia, N., Tiggeman, F., Borges, E., Lucca, G., Santos, H., & Dimuro, G. (2021). Exploring the Relationships between Data Complexity and Classification Diversity in Ensembles. *1*(Iceis), 652–659. http://dx.doi.org/10.5220/0010440006520659.

Han, H., Wang, W.-y., & Mao, B.-h. (2005). Borderline-SMOTE : A New Over-Sampling Method in. In *International conference on intelligent computing, ICIC 2005, Hefei, China, August 23-26, 2005, Proceedings, Part I* (pp. 878–887). URL: http://dx.doi.org/10.1007/11538059_91.

Ho, T. K. (1995). Random decision forests. *1*, In *Proceedings of 3rd international conference on document analysis and recognition* (pp. 278–282). IEEE.

Ho, T. K. (2008). Data complexity analysis: Linkage between context and solution in classification. In N. da Vitoria Lobo, T. Kasparis, F. Roli, J. T. Kwok, M. Georgiopoulos, G. C. Anagnostopoulos, & M. Loog (Eds.), *Structural, syntactic, and statistical pattern recognition* (p. 1). Berlin, Heidelberg: Springer Berlin Heidelberg.

Ho, T. K., & Baird, H. S. (1998). Pattern Classification with Compact Distribution Maps. *Computer Vision and Image Understanding*, *70*(1), 101–110. http://dx.doi.org/10.1006/cviu.1998.0624.

Ho, T. K., & Basu, M. (2002). Complexity measures of supervised classification problems. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *24*(3), 289–300. http://dx.doi.org/10.1109/34.990132.

Jain, A. K., Duin, R. P., & Mao, J. (2000). Statistical pattern recognition: A review. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *22*(1), 4–37. http://dx.doi.org/10.1109/34.824819.

Jielai, X., Hongwei, J., & Qiyi, T. (2015). Introduction to artificial neural networks. *Advanced Medical Statistics*, 1431–1449. http://dx.doi.org/10.1142/9789814583312_0037.

Kim, Y. S. (2010). Performance evaluation for classification methods: A comparative simulation study. *Expert Systems with Applications*, *37*(3), 2292–2306. http://dx.doi.org/10.1016/j.eswa.2009.07.043, URL: http://dx.doi.org/10.1016/j.eswa.2009.07.043.

Kinal, M., & Woźniak, M. (2020). Data preprocessing for des-knn and its application to imbalanced medical data classification. In N. T. Nguyen, K. Jearanaitanakij, A. Selamat, B. Trawiński, & S. Chittayasothorn (Eds.), *Intelligent information and database systems* (pp. 589–599). Cham: Springer International Publishing.

Ko, A. H., Sabourin, R., & Britto, Jr., A. S. (2008). From dynamic classifier selection to dynamic ensemble selection. *Pattern Recognition*, *41*(5), 1718–1731. http://dx.doi.org/10.1016/j.patcog.2007.10.015, URL: https://www.sciencedirect.com/science/article/pii/S0031320307004499.

Li, K., Kong, X., Lu, Z., Wenyin, L., & Yin, J. (2014). Boosting weighted ELM for imbalanced learning. *Neurocomputing*, *128*, 15–21. http://dx.doi.org/10.1016/j.neucom.2013.05.051, URL: http://dx.doi.org/10.1016/j.neucom.2013.05.051.

Liu, Z., Jin, W., & Mu, Y. (2020). Variances-constrained weighted extreme learning machine for imbalanced classification. *Neurocomputing*, *403*, 45–52. http://dx.doi.org/10.1016/j.neucom.2020.04.052, URL: http://dx.doi.org/10.1016/j.neucom.2020.04.052.

Liu, X. Y., Wu, J., & Zhou, Z. H. (2009). Exploratory undersampling for class-imbalance learning. *IEEE Transactions on Systems, Man and Cybernetics, Part B*, *39*(2), 539–550. http://dx.doi.org/10.1109/TSMCB.2008.2007853.

LLC, J. S. D. (2022). The Paired t-Test. [Online; accessed 26-Oct-2022].

Meyer, D., Leisch, F., & Hornik, K. (2003). The support vector machine under test. *Neurocomputing*, *55*(1–2), 169–186. http://dx.doi.org/10.1016/S0925-2312(03)00431-4.

Mousavi, S. M., & Langston, C. A. (2017). Automatic noise-removal/signal-removal based on general cross-validation thresholding in synchrosqueezed domain and its application on earthquake data. *Geophysics*, *82*(4), V211–V227. http://dx.doi.org/10.1190/GEO2016-0433.1.

Ougiaroglou, S., Nanopoulos, A., Papadopoulos, A., Manolopoulos, Y., & Welzer, T. (2007). Adaptive k-nearest-neighbor classification using a dynamic number of nearest neighbors. (pp. 66–82). http://dx.doi.org/10.1007/978-3-540-75185-4_7.

Pandis, N. (2015). Comparison of 2 means for matched observations (paired t test) and t test assumptions. *American Journal of Orthodontics and Dentofacial Orthopedics*, *148*(3), 515–516. http://dx.doi.org/10.1016/j.ajodo.2015.06.011, URL: https://www.sciencedirect.com/science/article/pii/S0889540615007490.

Quinlan, J. R. (1986). Induction of Decision Trees. *Machine Learning*, *1*(1), 81–106. http://dx.doi.org/10.1023/A:1022643204877.

Raghuwanshi, B. S., & Shukla, S. (2018). Class-specific extreme learning machine for handling binary class imbalance problem. *Neural Networks*, *105*, 206–217. http://dx.doi.org/10.1016/j.neunet.2018.05.011, URL: http://dx.doi.org/10.1016/j.neunet.2018.05.011.

Raghuwanshi, B. S., & Shukla, S. (2019). Class imbalance learning using UnderBagging based kernelized extreme learning machine. *Neurocomputing*, *329*, 172–187. http://dx.doi.org/10.1016/j.neucom.2018.10.056, URL: http://dx.doi.org/10.1016/j.neucom.2018.10.056.

Raghuwanshi, B. S., & Shukla, S. (2021). Minimum class variance class-specific extreme learning machine for imbalanced classification. *Expert Systems with Applications*, *178*(May 2020), Article 114994. http://dx.doi.org/10.1016/j.eswa.2021.114994, URL: http://dx.doi.org/10.1016/j.eswa.2021.114994.

Saez, J. A., Galar, M., & Krawczyk, B. (2019). Addressing the Overlapping Data Problem in Classification Using the One-vs-One Decomposition Strategy. *IEEE Access*, *7*, 83396–83411. http://dx.doi.org/10.1109/ACCESS.2019.2925300.

Schapire, R. E. (2013). Explaining adaboost. *Empirical Inference: Festschrift in Honor of Vladimir N. Vapnik*, 37–52. http://dx.doi.org/10.1007/978-3-642-41136-6_5.

Schneider, K.-M. (2003). A comparison of event models for Naive Bayes anti-spam e-mail filtering. (p. 307). http://dx.doi.org/10.3115/1067807.1067848.

Seiffert, C., Khoshgoftaar, T. M., Van Hulse, J., & Napolitano, A. (2010). RUSBoost: A hybrid approach to alleviating class imbalance. *IEEE Transactions on Systems, Man, and Cybernetics Part A:Systems and Humans*, *40*(1), 185–197. http://dx.doi.org/10.1109/TSMCA.2009.2029559.

Smith, S. P., & Jain, A. K. (1988). A Test to Determine the Multivariate Normality of a Data Set. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *10*(5), 757–761. http://dx.doi.org/10.1109/34.6789.

Tomek, I. (1976). Two Modifications of Cnn.. *IEEE Transactions on Systems, Man and Cybernetics*, *SMC-6*(11), 769–772. http://dx.doi.org/10.1109/TSMC.1976.4309452.

Verbaeten, S., & Van Assche, A. (2003). Ensemble methods for noise elimination in classification problems. In T. Windeatt, & F. Roli (Eds.), *Multiple classifier systems* (pp. 317–325). Berlin, Heidelberg: Springer Berlin Heidelberg.

Vuttipittayamongkol, P., Elyan, E., & Petrovski, A. (2021). On the class overlap problem in imbalanced data classification. *Knowledge-Based Systems*, *212*, Article 106631. http://dx.doi.org/10.1016/j.knosys.2020.106631, URL: http://dx.doi.org/10.1016/j.knosys.2020.106631.

Wallace, B. C., Small, K., Brodley, C. E., & Trikalinos, T. A. (2011). Class imbalance, redux. In *Proceedings - IEEE international conference on data mining, ICDM* (pp. 754–763). http://dx.doi.org/10.1109/ICDM.2011.33.

Wang, S., & Yao, X. (2009). Diversity analysis on imbalanced data sets by using ensemble models. In *2009 IEEE symposium on computational intelligence and data mining, CIDM 2009 - Proceedings* (pp. 324–331). http://dx.doi.org/10.1109/CIDM.2009.4938667.

Watson, J. C., Ho, C. M., & Boham, M. (2021). Advancing the Counseling Profession Through Intervention Research. *Journal of Counseling and Development*, *99*(2), 134–144. http://dx.doi.org/10.1002/jcad.12361.

Xiao, W., Zhang, J., Li, Y., Zhang, S., & Yang, W. (2017). Class-specific cost regulation extreme learning machine for imbalanced classification. *Neurocomputing*, *261*(2017), 70–82. http://dx.doi.org/10.1016/j.neucom.2016.09.120, URL: http://dx.doi.org/10.1016/j.neucom.2016.09.120.

Xiong, H., Li, M., Jiang, T., & Zhao, S. (2013). Classification algorithm based on NB for class overlapping problem. *Applied Mathematics & Information Sciences*, *7*(2 L), 409–415. http://dx.doi.org/10.12785/amis/072L05.

Yang, L. (2011). Classifiers selection for ensemble learning based on accuracy and diversity. *Procedia Engineering*, *15*, 4266–4270. http://dx.doi.org/10.1016/j.proeng.2011.08.800.

Yigit, H. (2013). A weighting approach for KNN classifier. *1*, In *2013 international conference on electronics, computer and computation, ICECCO 2013* (2), (pp. 228–231). http://dx.doi.org/10.1109/ICECCO.2013.6718270.

Zheng, J., Liu, Y., & Ge, Z. (2022). Dynamic ensemble selection based improved random forests for fault classification in industrial processes. *IFAC Journal of Systems and Control*, *20*, Article 100189. http://dx.doi.org/10.1016/j.ifacsc.2022.100189, URL: http://dx.doi.org/10.1016/j.ifacsc.2022.100189.

Zhou, Z. H., Wu, J., & Tang, W. (2010). Erratum: Ensembling neural networks: Many could be better than all (Artificial Intelligence (2002) 137:1-2 (239-263). *Artificial Intelligence*, *174*(18), 1570. http://dx.doi.org/10.1016/j.artint.2010.10.001, URL: http://dx.doi.org/10.1016/j.artint.2010.10.001.

Zong, W., Huang, G. B., & Chen, Y. (2013). Weighted extreme learning machine for imbalance learning. *Neurocomputing*, *101*, 229–242. http://dx.doi.org/10.1016/j.neucom.2012.08.010, URL: http://dx.doi.org/10.1016/j.neucom.2012.08.010.